



# CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

## BÀI 8: Quy hoạch động & Knapsack

# PHẦN 1



## DP là gì?

Dynamic programming

- Giải bài toán lớn bằng cách chia thành bài con.
- Các bài con trùng lặp  $\rightarrow$  ta lưu lại kết quả để dùng lại.

Nôm na:

□ “Tính rồi thì đừng tính lại, chỉ tra bảng.”

## Khi nào dùng DP?

Bài toán thường có:

- Overlapping subproblems: bài con giống nhau xuất hiện nhiều lần.
- Optimal substructure: lời giải tối ưu của bài lớn xây từ lời giải tối ưu bài nhỏ.

Ví dụ: fibonacci, climbing stairs, đường đi ngắn nhất, knapsack, LCS...

## Recursion bình thường

Ví dụ fibonacci:

$$F(0) = 0, F(1) = 1$$

$$F(n) = F(n-1) + F(n-2)$$

Code đệ quy:

```
def fib_rec(n):  
    if n <= 1:  
        return n  
    return fib_rec(n-1) + fib_rec(n-2)
```

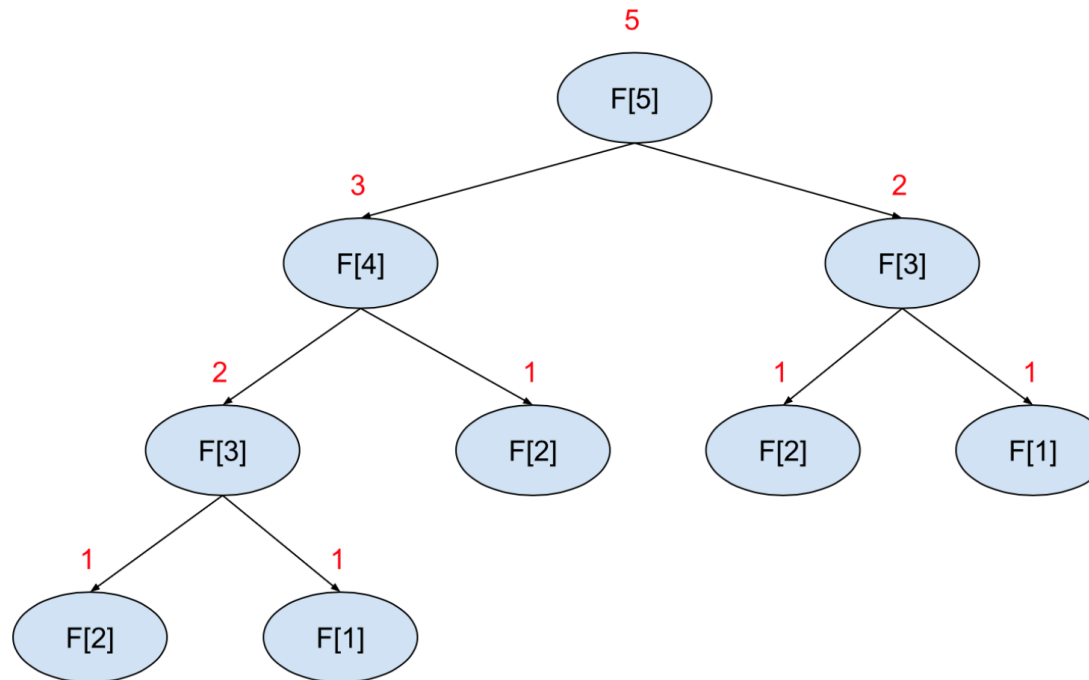
Nhược điểm:

- Rất nhiều lời gọi lặp lại.
- Thời gian  $\sim O(2^n)$ .

## Hình cây đệ quy fibonacci

Ví dụ  $n = 5$ :

- $F(5)$  gọi  $F(4)$  và  $F(3)$ .
- $F(4)$  lại gọi  $F(3)$  và  $F(2)$ .
- $F(3)$  xuất hiện nhiều lần.



Ta đang tính đi tính lại cùng một thứ.

DP = cách “có tổ chức” để không làm việc thừa này.

## Memoization (top-down)

Ý tưởng:

- Vẫn dùng recursion.
- Thêm 1 dict/mảng cache để lưu kết quả đã tính.

Thời gian giảm xuống  $O(n)$ .

```
def fib_memo(n, memo=None):  
    if memo is None:  
        memo = {}  
    if n in memo:  
        return memo[n]  
    if n <= 1:  
        memo[n] = n  
    else:  
        memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)  
    return memo[n]
```

## DP bottom-up (tabulation)

Thay vì gọi đệ quy:

- Ta tạo 1 mảng dp.
- Tính  $F(0)$ ,  $F(1)$ , rồi  $F(2)$ ,  $F(3)$ , ...,  $F(n)$  theo thứ tự.

Không dùng recursion, dễ debug hơn.

```
def fib_tab(n):  
    if n <= 1:  
        return n  
    dp = [0] * (n + 1)  
    dp[0] = 0  
    dp[1] = 1  
    for i in range(2, n+1):  
        dp[i] = dp[i-1] + dp[i-2]  
    return dp[n]
```



## So sánh 3 cách Fibonacci

- Độ quy thuần: code ngắn, nhưng rất chậm, exponential.
- Memoization: đệ quy + cache, nhanh, vẫn giữ style recursion.
- DP bottom-up: không đệ quy, dùng mảng, dễ nhìn thứ tự tính.

Tất cả cùng 1 công thức  $F(n) = F(n-1) + F(n-2)$ .

Khác nhau ở cách tổ chức tính.

## Bài toán: Climbing stairs

- Có  $n$  bậc thang.
- Mỗi lần bước được 1 hoặc 2 bậc.
- Hỏi có bao nhiêu cách khác nhau để lên đỉnh?

Ví dụ:

- $n=2 \rightarrow 2$  cách:  $(1+1)$ ,  $(2)$ .
- $n=3 \rightarrow 3$  cách:  $(1+1+1)$ ,  $(1+2)$ ,  $(2+1)$ .

## Nhìn Climbing stairs như Fibonacci

Gọi  $\text{ways}[n]$  = số cách lên  $n$  bậc.

Quan sát:

Để đứng ở bậc  $n$ :

- Bước từ bậc  $n-1$  (thêm 1 bước).
- Hoặc từ bậc  $n-2$  (thêm 2 bước).

→  $\text{ways}[n] = \text{ways}[n-1] + \text{ways}[n-2]$ .

Giống hệt Fibonacci, chỉ khác base case.

## DP cho climbing stairs

Base:

- $\text{ways} = 1$  (1 cách “đứng tại chỗ”).
- $\text{ways} = 1$  (chỉ có 1 cách: bước 1).

Transition:

- $\text{ways}[n] = \text{ways}[n-1] + \text{ways}[n-2]$ .

```
def climb_stairs(n):  
    if n <= 1:  
        return 1  
  
    dp = [0] * (n + 1)  
    dp[0] = 1  
    dp[1] = 1  
    for i in range(2, n+1):  
        dp[i] = dp[i-1] + dp[i-2]  
    return dp[n]
```

## Tư duy “state” và “transition”

Các bước tổng quát của DP:

1. Chọn state:  $dp[i]$  nghĩa là gì?

- Fibonacci:  $dp[i] = F(i)$ .
- Stairs:  $dp[i] =$  số cách lên  $i$  bậc.

2. Tìm transition:

- $dp[i]$  từ  $dp[i-1]$ ,  $dp[i-2]$ ?

3. Chọn base case:  $dp$ ,  $dp$  là gì?

## Tối ưu bộ nhớ dp 1D

Fibonacci / stairs: Mỗi bước chỉ cần 2 giá trị trước đó.

Có thể không cần mảng, chỉ dùng 2 biến:

```
def climb_stairs_optimized(n):  
    if n <= 1:  
        return 1  
    prev2 = 1 # dp[0]  
    prev1 = 1 # dp[1]  
    for i in range(2, n+1):  
        cur = prev1 + prev2  
        prev2 = prev1  
        prev1 = cur  
    return prev1
```

Bộ nhớ:  $O(1)$ .

## **DP vs brute force: cảm giác**

Brute force (recursion thuần) thường:

- “Thử mọi cách”, nhiều nhánh, cây gọi hàm rất lớn.

DP:

- Thay vì thử lại, ta nhớ kết quả của từng bước trung gian.
- Như việc giải bài toán một lần, sau đó chỉ tra bảng.

## Nhận diện bài DP

Khi gặp bài mới, thử hỏi:

1. Có thể chia bài lớn thành bài con giống nhau không?
2. Kết quả bài lớn có ghép từ kết quả bài con không?
3. Bài con có lặp lại nhiều lần không?

Nếu “có”  $\rightarrow$  rất có thể là bài DP.



## Ba cách giải 1 bài

Với 1 bài kiểu fibonacci:

- Cách 1: recursion thuần  $\rightarrow$  dễ nghĩ, chậm.
- Cách 2: recursion + memo (top-down)  $\rightarrow$  tối ưu dần.
- Cách 3: DP bottom-up (tabulation)  $\rightarrow$  iterative, dễ kiểm soát hơn.

Trong môn này, chúng ta sẽ ưu tiên style 2 và 3.

# PHẦN 2



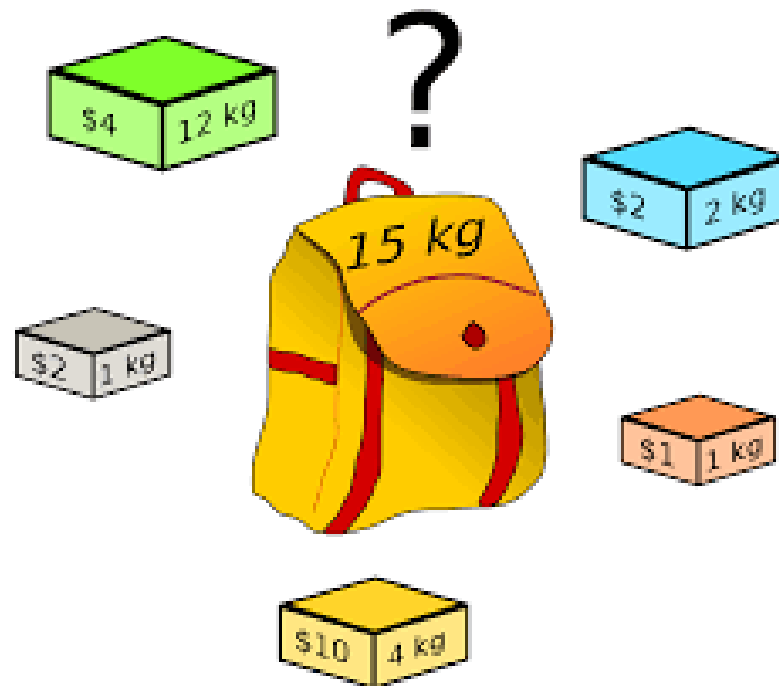
## Bài toán knapsack 0/1

Cho  $n$  món đồ, mỗi món  $i$  có:

- weight:  $wt[i]$
- value:  $val[i]$
- Balo sức chứa tối đa  $W$ .

Mỗi món: chỉ chọn 0 hoặc 1 lần (0/1).

Mục tiêu: chọn tập món có tổng  $weight \leq W$ , tổng value lớn nhất.



Ví dụ:

$W = 5$ .

- Item 0:  $wt=1$ ,  $val=1$ .
- Item 1:  $wt=2$ ,  $val=4$ .
- Item 2:  $wt=3$ ,  $val=5$ .

Câu hỏi:

- Nên chọn item nào để value lớn nhất mà không vượt quá  $W=5$ ?

## **Bảng DP 2 chiều**

Định nghĩa:  $dp[i][w]$  = max value dùng  $i$  món đầu  $(0..i-1)$ , sức chứa  $w$ .

Kích thước:

- Hàng:  $0..n$ .
- Cột:  $0..W$ .

Base:

- $dp[w] = 0$  (không có món).
- $dp[i] = 0$  (sức chứa 0).

## Công thức chuyển dp 2D

Xét item  $i$  (index trong values là  $i-1$ ) và capacity  $w$ :

- Nếu  $\text{weight}[i-1] > w \rightarrow$  không lấy được:  $\text{dp}[i][w] = \text{dp}[i-1][w]$ .
- Ngược lại có 2 lựa chọn:
  - Không lấy:  $\text{dp}[i-1][w]$ .
  - Lấy:  $\text{value}[i-1] + \text{dp}[i-1][w - \text{weight}[i-1]]$ .

Chọn max:  $\text{dp}[i][w] = \max(\text{dp}[i-1][w], \text{value}[i-1] + \text{dp}[i-1][w - \text{weight}[i-1]])$

## Code xây bảng dp 2D

```
def knapsack_dp_table(values, weights, W):  
    n = len(values)  
    dp = [[0] * (W + 1) for _ in range(n + 1)]  
  
    for i in range(1, n + 1):  
        wt = weights[i - 1]  
        val = values[i - 1]  
        for w in range(1, W + 1):  
            if wt > w:  
                dp[i][w] = dp[i - 1][w]  
            else:  
                dp[i][w] = max(  
                    dp[i - 1][w],  
                    val + dp[i - 1][w - wt]  
                )  
  
    return dp
```

Kết quả tối đa sẽ nằm ở  $dp[n][W]$ .

## Demo in bảng dp

```
def demo_dp_table():  
    values = [1, 4, 5]  
    weights = [1, 2, 3]  
    W = 5  
  
    dp = knapsack_dp_table(values, weights, W)  
  
    print("Bảng dp (hàng: i, cột: w):")  
    for row in dp:  
        print(row)  
  
    print("Max value:", dp[len(values)][W])
```

- Mỗi khi cho thêm item, các ô  $dp[i][w]$  không giảm.
- Cột  $w$  nhỏ không thể chứa item quá nặng.



## Truy vết nghiệm từ bảng 2D

Mục tiêu: Biết item nào được chọn.

Ý tưởng:

- Bắt đầu từ  $(i = n, w = W)$ .
- Nếu  $dp[i][w] == dp[i-1][w] \rightarrow$  item  $i-1$  không được chọn.
- Nếu khác  $\rightarrow$  item  $i-1$  được chọn, trừ  $w$ :  $w -= \text{weight}[i-1]$ .
- Giảm  $i$  dần đến 0.

## Code truy vết items (2D)

```
def trace_items_from_dp(dp, values, weights, W):  
    n = len(values)  
    w = W  
    chosen = []  
  
    i = n  
    while i > 0 and w > 0:  
        if dp[i][w] == dp[i - 1][w]:  
            # Không chọn item i-1  
            i -= 1  
        else:  
            # Chọn item i-1  
            chosen.append(i - 1)  
            w -= weights[i - 1]  
            i -= 1  
  
    chosen.reverse()  
    return chosen
```

chosen là danh sách index item được chọn.

## Demo full: max value + nghiệm

Tổng weight không vượt quá W.

Tổng value = max\_val.

```
def demo_knapsack_2d_trace():  
    values = [1, 4, 5]  
    weights = [1, 2, 3]  
    W = 5  
  
    dp = knapsack_dp_table(values, weights, W)  
    max_val = dp[len(values)][W]  
    chosen = trace_items_from_dp(dp, values, weights, W)  
  
    print("Max value:", max_val)  
    print("Chọn các item (index):", chosen)  
    total_weight = sum(weights[i] for i in chosen)  
    print("Tổng weight:", total_weight)  
    for i in chosen:  
        print(f"    item {i}: wt={weights[i]}, val={values[i]}")
```

## Tối ưu xuống DP 1 chiều

Nhận xét: Để tính  $dp[i][w]$ , ta chỉ dùng hàng  $i-1$ .

- Có thể dùng 1 mảng  $dp[w]$ :  $dp[w] = \text{max value}$  với sức chứa  $w$ , xét đến các item đã xử lý.
- Cập nhật  $dp$  cho từng item: Duyệt  $w$  từ  $W \rightarrow 0$  để không dùng lại item nhiều lần.

## Công thức DP 1 chiều

Với mỗi item i:

```
for w in range(W, weights[i] - 1, -1):  
    dp[w] = max(  
        dp[w],                # không lấy  
        values[i] + dp[w - weights[i]] # lấy item i  
    )
```

Tại sao duyệt w giảm dần? Đảm bảo  $dp[w - \text{weights}[i]]$  vẫn là kết quả trước khi dùng item i.

Nếu duyệt tăng dần  $\rightarrow$  có thể dùng item i nhiều lần  $\rightarrow$  sai 0/1.

## Code knapsack DP 1D

```
def knapsack_1d(values, weights, W):  
    n = len(values)  
    dp = [0] * (W + 1)  
  
    for i in range(n):  
        wt = weights[i]  
        val = values[i]  
        for w in range(W, wt - 1, -1):  
            dp[w] = max(dp[w], val + dp[w - wt])  
  
    return dp[W]
```

- Time:  $O(n*W)$  như 2D.
- Memory: từ  $O(n*W)$  xuống  $O(W)$ .

## DP 1D + bảng choice

Một kỹ thuật thực dụng:

- Vẫn dùng dp 1D:  $dp[w]$ .
- Thêm bảng  $choice[i][w]$ : True nếu tại item  $i$  và capacity  $w$ , ta lấy item  $i$ .
- Khi update  $dp[w]$ , nếu lấy item  $i$  tốt hơn  $\rightarrow$  set  $choice[i][w] = \text{True}$ .

Truy vết:

- Bắt đầu từ  $i = n-1$ ,  $w = W$ .
- Nếu  $choice[i][w] = \text{True} \rightarrow$  chọn  $i$ , giảm  $w$ .
- Giảm  $i$  dần.

## Code minh họa choice

```
def knapsack_1d_with_choice(values, weights, W):  
    n = len(values)  
    dp = [0] * (W + 1)  
    choice = [[False] * (W + 1) for _ in range(n)]  
  
    for i in range(n):  
        wt = weights[i]  
        val = values[i]  
        for w in range(W, wt - 1, -1):  
            take_val = val + dp[w - wt]  
            if take_val > dp[w]:  
                dp[w] = take_val  
                choice[i][w] = True  
    return dp, choice
```



# Truy vết từ choice

```
def trace_from_choice(values, weights, W):  
    dp, choice = knapsack_1d_with_choice(values, weights, W)  
    n = len(values)  
    w = W  
    chosen = []  
  
    i = n - 1  
    while i >= 0 and w > 0:  
        if choice[i][w]:  
            chosen.append(i)  
            w -= weights[i]  
        i -= 1  
  
    chosen.reverse()  
    return dp[W], chosen
```



# Kết thúc